Day 2 — Group Exercise: Branch, Pull Request, Merge Conflict

Team size: 3,
Time: 30 minutes

Description

Your team maintains a tiny command-line app called **Team Toolbox**. Each of you adds one tool. The catch: all three of you have to edit **the same two files** — `toolbox.py` and `README.md` — so git cannot merge everything for you.

That is the whole point. You will produce merge conflicts on purpose, and resolve them.

You will all work in **one shared repository**, and each will work on a branch. This is how most teams work day to day: one repo, everyone a collaborator, nobody commits straight to `main`.

Roles — decide these in the first 60 seconds

Role	Who	Job	Branch	Tool to add
Repo owner	A	creates the repo, reviews and merges every PR	`feature/reverse`	`reverse`
Contributor	B	PRs second	`feature/wordcount`	`wordcount`
Contributor	C	PRs third	`feature/initials`	`initials`

Merge order is fixed: A, then B, then C. A merges clean. B creates a conflict. C creates a bigger one. Everyone sees a different versions of the same files.

> **One rule for the whole exercise: nobody pushes to `main`.** Every change reaches `main` through a pull request — including A's.

Which half am I in?

You will keep switching between your laptop and github.com. Losing track of which is the #1 way to get stuck. Here is the whole exercise on one axis:

#	Who	Where	What happens
1	A	laptop	push the starter code
2	A	github	Settings → Collaborators → add B and C
3	B, C	github	accept the invitation
4	all	laptop	clone, branch, edit, commit
5	all	laptop	`git push origin feature/<tool>`

#	Who	Where	What happens
6	all	github	open a PR: base `main` ← compare your branch
7	A	github	**Merge pull request** on PR #1
8	B, C	github	your PR now says *conflicts must be resolved*
9	B, C	laptop	`git fetch` or `git pull` → `git merge` → resolve → commit
10	B, C	laptop	`git push` — **the PR updates itself**
11	A	github	PR changes to *able to merge* → merge it
12	all	laptop	`git pull origin main`

Step 10 is the one to notice: you do not edit the pull request. You push to a branch, and the PR — which is only a *pointer* to that branch — re-checks itself.

Step 0 — Set up (3 min)

Member A only

1. On GitHub, create a **public** repo named `dsai692_pr_example`. Do **not** add a README.
2. Copy and paste `Day2/dsai692_pr_example` to somewhere else, and `cd` to that folder:

```
git init -b main
git add .
git commit -m "Initial commit: starter toolbox"
git remote add origin https://github.com/<A-
username>/dsai692_pr_example.git
git push -u origin main
```

3. Add your teammates: repo → **Settings** → **Collaborators** → **Add people** → their GitHub usernames.
4. Share the repo URL with your teammates.

Members B and C

1. **Accept the invitation.** It arrives by email, and it is also waiting at `https://github.com/<A-username>/dsai692_pr_example/invitations`. If you skip this, your first push will fail.
2. Clone the shared repo:

```
git clone https://github.com/<A-username>/dsai692_pr_example.git
cd dsai692_pr_example
git remote -v
```

You should see two lines, both `origin`, both pointing at A's repo. **You all share one remote.** Your branch is what keeps your work separate — not a separate repo.

Everyone — check it runs

```
python toolbox.py shout hello world     # -> HELLO WORLD!
```

Step 1 — Make your branch and your change (5 min)

Everyone works **at the same time**. Do not wait for each other.

```
git switch -c feature/<your-tool>      # e.g. git switch -c feature/wordcount
```

Now make **four** edits.

1. Create your tool file — `tools/<your_tool>.py`. Nobody else touches this file.

Member A — `tools/reverse.py`

```python
"""reverse tool."""


def reverse(text):
    """Return TEXT reversed."""
    return text[::-1]
```

Member B — `tools/wordcount.py`

```python
"""word_count tool."""


def word_count(text):
    """Return the number of words in TEXT."""
    return str(len(text.split()))
```

Member C — `tools/initials.py`

```python
"""initials tool."""


def initials(text):
    """Return the initials of each word in TEXT."""
    return "".join(w[0].upper() for w in text.split() if w)
```

2. In `toolbox.py`, add your import at the END of the import block:

```
# --- IMPORT BLOCK ---------------------------------------------------
# Add your import at the END of this block, on the line above the dashes.
from tools.shout import shout
from tools.wordcount import word_count      # <-- your line goes last
# --------------------------------------------------------------------
```

3. In `toolbox.py`, add your tool at the END of the registry:

```
TOOLS = {
    "shout": shout,
    "wordcount": word_count,                # <-- your line goes last
}
```

4. In `README.md`, add one row at the BOTTOM of the Tools table, and add a file `notes/<your-github-username>.md` with one line about what you just learned.

Then test, commit, push:

```
python toolbox.py <your-tool> hello world
git add .
git commit -m "Add <your-tool> tool"
git push origin feature/<your-tool>
```

Refresh the repo on GitHub — you should now see all three branches in the branch dropdown. Same repo, three parallel lines of work.

Step 2 — Open your pull requests (3 min)

All three of you, on GitHub, in the shared repo:

1. After your push, the repo home page shows a yellow banner: **"feature/... had recent pushes"** → click **Compare & pull request**. (If not, **Pull requests** tab → **New pull request**.)
2. Check the two dropdowns read: **base:** `main` ← **compare:** `feature/<your-tool>`
3. Add a title and description → **Create pull request**.

Click through the four tabs once so you know what's there:

Tab	What's in it
Conversation	your description, comments, and the merge box at the bottom
Commits	the commits on your branch
Checks	automated tests (empty here — real projects run CI)

Tab	What's in it
Files changed	the diff your reviewer reads

Use this PR description template:

```
## What
Adds the `<tool>` tool.

## Why
Each teammate contributes one tool to the shared registry.

## How to test
python toolbox.py <tool> hello world

## Touched shared files
toolbox.py (import block + registry), README.md (tools table)
```

Scroll to the bottom of the **Conversation** tab. That box is the whole exercise:

```
clean:        This branch has no conflicts with the base branch
                [ Merge pull request ]

conflicted:   This branch has conflicts that must be resolved
                Conflicting files:  toolbox.py   README.md
                [ Resolve conflicts ]
```

A's PR says "no conflicts." B's and C's will flip to "conflicts must be resolved" once A's PR is merged. That message is your cue for Step 4.

Step 3 — Merge PR #1 (2 min)

A, on PR #1:

1. Review the **Files changed** tab.
2. Click the ▾ next to **Merge pull request** and look at all three options:
   - **Create a merge commit** - use this one
   - **Squash and merge** — collapses your commits into one; the branch vanishes from the graph, and Step 6 has nothing to show
   - **Rebase and merge** — keeps the commits but rewrites them, no merge commit
   - For more details, I recommend to watch this Youtube 🎬 .
3. Pick **Create a merge commit** → **Confirm merge**.
4. On the purple "successfully merged" banner, leave **Delete branch** alone for now — you'll do cleanup in Step 6.

Everyone: refresh B's and C's PR pages. They are now marked as conflicted.

> **Two separate copies.** A merged on **github.com**. Each member's *laptop* has no idea — `git log` there still shows the old commit until he/she pulls. That is what created B's and C's conflicts, and it is why Step 4 starts with `fetch`.

> ⚠ **B and C: you will see a "Resolve conflicts" button on your PR. Don't click it.** GitHub can resolve simple conflicts in the browser, but today the point is to do it in the terminal, where you can actually see what git is doing.

Step 4 — B resolves the conflict (7 min)

B, in your local clone:

```
git checkout feature/wordcount
git fetch origin
git merge origin/main
```

Note : `git fetch` only downloads remote changes to your local repository without altering your code, while `git pull` downloads those changes and automatically merges them into your active working files

git stops and tells you exactly what it could not decide:

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Auto-merging toolbox.py
CONFLICT (content): Merge conflict in toolbox.py
Automatic merge failed; fix conflicts and then commit the result.
```

```
git status --short
```

```
UU README.md
A  notes/ana-lead.md
UU toolbox.py
A  tools/reverse.py
```

> **Read this before you fix anything.** `UU` = both of you updated it, git needs *you*. `A` = added cleanly, no help needed. A's new file `tools/reverse.py` and their note merged with zero fuss, because nobody else touched them. **Conflicts come from shared lines, not from shared projects.**

Open `toolbox.py`. You will see:

```
from tools.shout import shout
<<<<<<< HEAD
```

```
from tools.wordcount import word_count
=======
from tools.reverse import reverse
>>>>>>> origin/main
```

- `<<<<<<< HEAD` … `=======` is **your** version (the branch you are on).
- `=======` … `>>>>>>> origin/main` is **their** version (what you are merging in).

Here the right answer is **keep both** — the two tools are not in competition. Delete the above three marker lines (`<<<<<<<`,`>>>>>>>`,`=======`) and keep both code lines:

```
from tools.shout import shout
from tools.reverse import reverse
from tools.wordcount import word_count
```

Do the same for the `TOOLS` registry and for the README table row. Then:

```
grep -rn "<<<<<<<\|>>>>>>>" .      # must print nothing
python toolbox.py                  # tools: reverse, shout, wordcount
python toolbox.py wordcount a b c  # 3
git add toolbox.py README.md
git commit -m "Merge origin/main into feature/wordcount; keep both tools"
git push origin feature/wordcount
```

Refresh the PR on GitHub — it flips back to **"Able to merge."** You never touched the PR itself; you fixed your branch and the PR followed.

A then merges PR #2.

Step 5 — C resolves a three-way conflict (5 min)

C, same commands — but now `origin/main` contains **two** tools, so your conflict has two lines on the other side:

```
from tools.shout import shout
<<<<<<< HEAD
from tools.initials import initials
=======
from tools.reverse import reverse
from tools.wordcount import word_count
>>>>>>> origin/main
```

Same resolution: keep everything, drop the markers, verify, commit, push.

```
git fetch origin
git merge origin/main
# ... resolve toolbox.py and README.md ...
grep -rn "<<<<<<<\|>>>>>>>" .
python toolbox.py initials data science and ai      # DSAA
git add .
git commit -m "Merge origin/main into feature/initials; keep all three
tools"
git push origin feature/initials
```

A merges PR #3.

> **Escape:** if the resolution goes sideways, `git merge --abort` puts you back exactly where you were before the merge. Nothing is lost. Start again.

Step 6 — Sync, verify, clean up (5 min)

Everyone:

```
git switch main
git pull origin main
python toolbox.py
```

Expected:

```
Team Toolbox
usage: python toolbox.py <tool> <text>
tools: initials, reverse, shout, wordcount
```

Look at the history you built:

```
git log --graph --oneline --all
```

Delete the branches you no longer need — two ways, same effect:

```
git branch -d feature/<your-tool>          # your laptop
git push origin -d feature/<your-tool>     # the shared repo
```

or on **github.com**: open your merged PR → **Delete branch** on the purple banner. (Changed your mind? The same spot offers **Restore branch**.)

Since you share one repo, deleting a remote branch deletes it **for everyone**. Only delete your own, and only after your PR is merged.

Done when

- [ ] `main` has all 4 tools and runs
- [ ] 3 PRs merged, in order
- [ ] No `<<<<<<<`, `>>>>>>>`, `=======` anywhere in the repo
- [ ] `notes/` has one file per teammate
- [ ] Feature branches deleted
- [ ] Nobody ever pushed directly to `main`
- [ ] Everyone can explain what `HEAD` meant in their own conflict

---

Command reference

Goal	Command
Create + switch to a branch	`git switch -c feature/x`
Switch to an existing branch	`git switch main`
Where am I?	`git branch` (star marks current)
Send my branch to the shared repo	`git push origin feature/x`
Get the latest, don't merge yet	`git fetch origin`
Bring `main` into my branch	`git merge origin/main`
See what conflicted	`git status --short` (`UU` = needs you)
Undo a merge in progress	`git merge --abort`
Finish a resolved merge	`git add <files>` then `git commit`
Update my local `main`	`git pull origin main`
See the branch structure	`git log --graph --oneline --all`
Delete branch, local / shared	`git branch -d x` / `git push origin -d x`